3.5 Localized Views

Let's say you have a *BooksController* in your application. Your *index* action renders content in `app/views/books/index.html.erb` template. When you put a *localized variant* of this template: `index.es.html.erb` in the same directory, Rails will render content in this template, when the locale is set to `:es`. When the locale is set to the default locale, the generic `index.html.erb` view will be used. (Future Rails versions may well bring this *automagic* localization to assets in `public`, etc.)
You can make use of this feature, e.g. when working with a large amount of static content, which would be clumsy to put inside YAML or Ruby dictionaries. Bear in mind, though, that any change you would like to do later to the template must be propagated to all of them.

3.6 Organization of Locale Files

When you are using the default SimpleStore shipped with the i18n library, dictionaries are stored in plain-text files on the disc. Putting translations for all parts of your application in one file per locale could be hard to manage. You can store these files in a hierarchy which makes sense to you.

For example, your `config/locales` directory could look like this:

```
|-defaults
|---es.rb
|---en.rb
|-models
|---book
|-----es.rb
|-----en.rb
|-views
|---defaults
|-----es.rb
|-----en.rb
|---books
|-----es.rb
|-----en.rb
|---users
|-----es.rb
|-----en.rb
|---navigation
|-----es.rb
|-----en.rb
```

This way, you can separate model and model attribute names from text inside views, and all of this from the "defaults" (e.g. date and time formats). Other stores for the i18n library could provide different means of such separation.

The default locale loading mechanism in Rails does not load locale files in nested dictionaries, like we have here. So, for this to work, we must explicitly tell Rails to look further:

```
# config/application.rb
config.i18n.load_path += Dir[Rails.root.join('config', 'locales',
'**', '*.{rb,yml}')]
```

4 Overview of the I18n API Features

You should have good understanding of using the i18n library now, knowing all necessary aspects of internationalizing a basic Rails application. In the following chapters, we'll cover it's features in more depth.

These chapters will show examples using both the `I18n.translate` method as well as the `translate` view helper method (noting the additional feature provide by the view helper method).